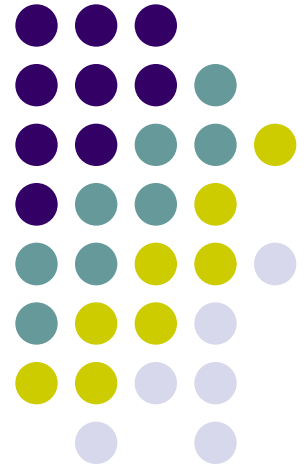
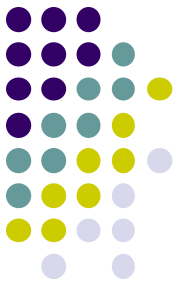


Natural Language Processing

CPSC 488

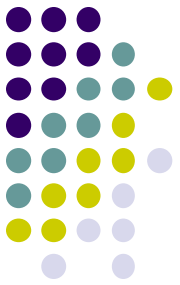
Text Processing and Classification





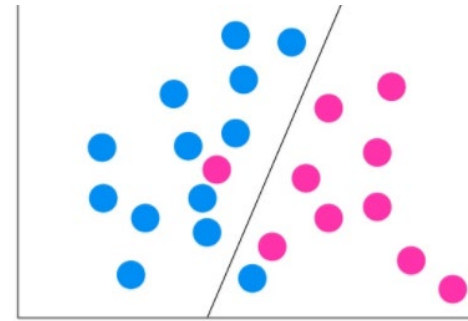
Lecture Overview

- Text processing for NLP tasks
 - Text processing and normalization
 - Vectorizing text data
- Text classification and its applications
 - Text classification and its applications
 - Sentiment analysis
 - Some issues of text classification



Text Classification

Text Classification

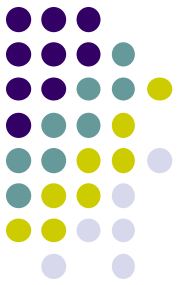


- **Text classification**

- Given a **training data set** $\{(X_1, \dots, X_k, y_j) | j = 1 \dots N\}$
- Learn a **function** or **classifier** $f: X \rightarrow y$ through the training
 - X are a set of all **documents** and y are **categories**.
 - The learned **classifier** f is used to determine a class $\hat{y}$.

- **Many applications**

- Spam detection
- Author or recipient identification, email filtering
- Document, topic, genre, or language categorization
- Sentiment analysis
 - Movies or customer ratings
 - Political opinion mining
- Information extraction
- Etc.



Spam Email Detection

- Is this email Spam or Ham?

Subject: Important notice!

From: Stanford University <newsforum@stanford.edu>

Date: October 28, 2011 12:34:16 PM PDT

To: undisclosed-recipients;;

Greats News!

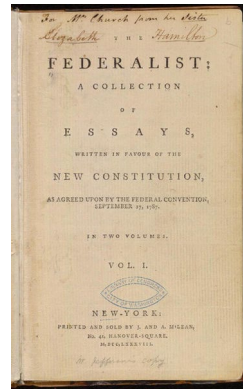
You can now access the latest news by using the link below to login to Stanford University News Forum.

<http://www.123contactform.com/contact-form-StanfordNew1-236335.html>

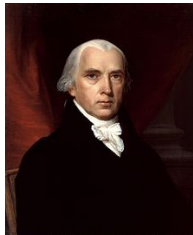
Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information about the new services.

© Stanford University. All Rights Reserved.

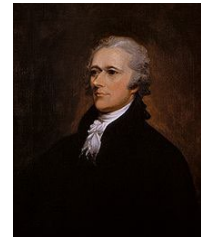
Authorship Identification



- *Who wrote which federalist papers?*
 - 1787-8: anonymous essays try to convince New York to ratify U.S Constitution: Jay, Madison, Hamilton.
 - Authorship of 12 of the letters in dispute
 - 1963: solved by Mosteller and Wallace using Bayesian methods



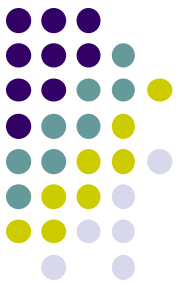
James Madison



Alexander Hamilton

- *Why matter?*
 - Need to know the writing style, context, other background information

Assigning Subject Categories, Genres, or Topics



● MEDLINE Article

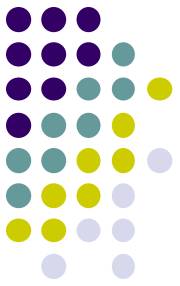


What is the subject of this medical article?

MeSH Subject Category Hierarchy
(by the National Medical Library)

- Antagonists and Inhibitors
- Blood Supply
- Chemistry
- Drug Therapy
- Embryology
- Epidemiology
- ...

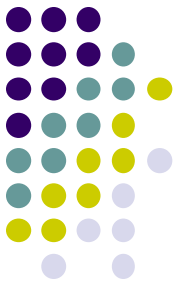
Who may find it useful?



Language Identification

- **Language identification**
 - Determining the language of source document or speech
- **Need in preprocessing tasks** for text document processing and speech recognition
 - During text parsing or indexing, e.g., words ending in “eux” tend to be French, in “o” tend to be Spanish.
 - Distinguishing closely related or similar languages
 - Speaker recognition

Sentiment Analysis



- Sentiment analysis is the **detection of attitudes**.
 - To predict the opinion expressed in a piece of **text** such as **Positive** (+) or **Negative** (–) rating, often based on sentiment words or lexicons
 - Example sentiment analysis results

positive

"... is a film which still causes real, not figurative, chills to run along my spine, and it is certainly the bravest and most ambitious fruit of Coppola's genius"

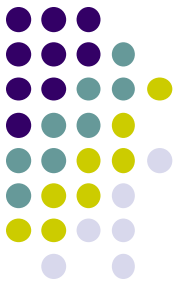
Roger Ebert, Apocalypse Now

- "I hated this movie. Hated hated hated hated hated this movie. Hated it. Hated every simpering stupid vacant audience-insulting moment of it. Hated the sensibility that thought anyone would like it."

negative

Roger Ebert, North

- **Sentiment words**
 - Words that express sentiment (positive, negative, neutral, etc.)
- **Sentiment lexicon**
 - A curated collection of sentiment words with extra information (e.g., scores)
 - SentiWordNet, AFINN lexicon, NRC emotion lexicon, MPQA subjectivity lexicon, etc.



Example Sentiment Words

Positive:

absolutely	beaming	calm
adorable	beautiful	celebrated
accepted	believe	certain
acclaimed	beneficial	champ
accomplish	bliss	champion
achieve	bountiful	charming
action	bounty	cheery
active	brave	choice
admire	bravo	classic
adventure	brilliant	classical
affirm	bubbly	clean
...		...

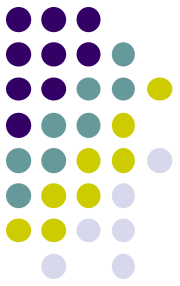
Negative:

abysmal	bad	callous
adverse	banal	can't
alarming	barbed	clumsy
angry	belligerent	coarse
annoy	bemoan	cold
anxious	beneath	collapse
apathy	boring	confused
appalling	broken	contradictory
atrocious		contrary
awful		corrosive
		corrupt
		...

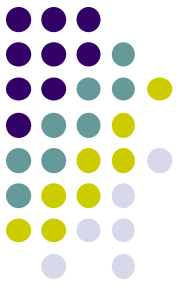
From <http://www.enchantedlearning.com/wordlist/>

Scherer Typology of Affective States

(A framework for understanding and categorizing emotions and affective experiences by K.R Scherer, 1943)



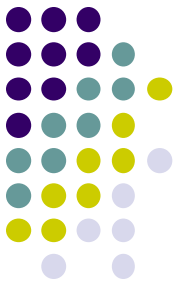
- **Emotion:** brief organically synchronized ... evaluation of a major event
 - *angry, sad, joyful, fearful, ashamed, proud, elated*
- **Mood:** diffuse non-caused low-intensity long-duration change in subjective feeling
 - *cheerful, gloomy, irritable, listless, depressed, buoyant*
- **Interpersonal stances:** affective stance toward another person in a specific interaction
 - *friendly, flirtatious, distant, cold, warm, supportive, contemptuous*
- **Attitudes:** enduring, affectively colored beliefs, dispositions towards objects or persons
 - *liking, loving, hating, valuing, desiring*
- **Personality traits:** stable personality dispositions and typical behavior tendencies
 - *nervous, anxious, reckless, morose, hostile, jealous*



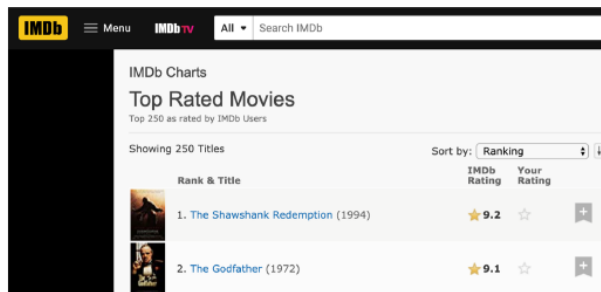
Why Sentiment Analysis?

- **Movie:** Is this review positive or negative?
- **Products:** What do people think about the new iPhone?
- **Public sentiment:** How is consumer confidence?
- **Politics:** What do people think about this candidate or issue?
- **Prediction:** Predict election outcomes or market trends from sentiment.

Movie Review Ratings



- Example movie review (sentiment)



positive

“... is a film which still causes real, not figurative, chills to run along my spine, and it is certainly the bravest and most ambitious fruit of Coppola's genius”

Roger Ebert, Apocalypse Now

- “I hated this movie. Hated hated hated hated hated this movie. Hated it. Hated every simpering stupid vacant audience-insulting moment of it. Hated the sensibility that thought anyone would like it.”

negative

Roger Ebert, North

- Example movie ratings

★★

Filled with horrific dialogue, laughable characters, a laughable plot, and really no interesting stakes during this film, "Star Wars Episode I: The Phantom Menace" is not at all what I wanted from a film that is supposed to be the huge opening to the segue into the fantastic Original Trilogy. The positives include the score, the sound effects, and most of the

KJ Proulx (/user/id/896976177/)

★ Super Reviewer

★★★★★

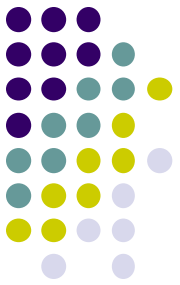
Extraordinarily faithful to the tone and style of the originals, The Force Awakens brings back the Old Trilogy's heart, humor, mystery, and fun. Since it is only the first piece in a new three-part journey it can't help but feel incomplete. But everything that's already there, from the stunning visuals, to the thrilling action sequences, to the charismatic new characters,

Matthew Samuel Mirliani (/user/id/896467979/)

★ Super Reviewer

Ratings require
quantification of
positive or negative
responses.

Political Opinion Mining



 **emilia** @PoliticalEmilia · 43m
As somebody whose immediate family are **immigrants** from Iran, I want to remind that this isn't the fault of Iranian Americans. Most of us want no more war in the Middle East.

Take your anger out at your government leaders, not at us. We have nothing to do with it. [#IranAttacks](#)

81 239 1.9K

 **Nithya Raman** @nithyavraman · Jan 6
LA is one of the most **immigrant**-rich cities in the US.

Almost 50% of residents are foreign-born. 10% are undocumented.

As Trump works to implement his racist agenda, what are our elected officials doing to defend **immigrant** Angelenos?

The answer: infuriatingly little. (thread)

55 138 606

 **Brigitte Gabriel** @ACTBrigitte · 3m
Thank Goodness there were ZERO U.S. casualties from the attacks Iran made tonight.

President **Trump** is monitoring the situation with his top leaders right now.

I've never felt more comfortable with a leader at the helm, than I do tonight with President **Trump** in office.

21 145 413

 **Palmer Report** @PalmerReport · 1m
So a foreign nation fired missiles at U.S. troops tonight, and the President of the United States ISN'T addressing the nation? How far gone is Donald **Trump**? His handlers don't even trust him to read a speech off a teleprompter anymore.

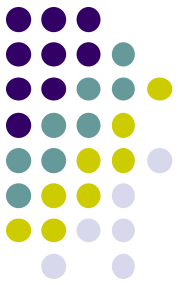
15 74 225

 **Andrea Chalupa** @AndreaChalupa · 7m
Trump is betting on Iran doing something so horrific to Americans that we rally around the flag, and the 2020 election becomes a mindless debate of who's "patriotic" vs. who's anti-war ("weak" on Iran).

47 147 425

Is positive or negative opinion?

More Sentiment Analysis



- **Targeted sentiment analysis**

- What is the author's attitude towards several different entities?
 - The vodka was good, but the meat was rotten.

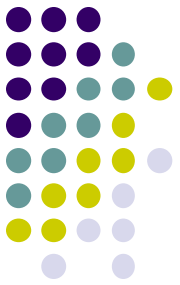
- **Emotion classification**

- Given a set of possible emotional states, which are expressed by the text?

- **Toxicity detection**

- The task of detecting hate speech, abuse, harassment, or other kinds of toxic language

More Text Classification Applications



- **Information Extraction (IE)**

- Automatically extracting structured information from text, specific piece of information, “facts”

- Examples

- Gather earnings, profits, board members, headquarters from company reports

“The headquarters of BHP Biliton limited, and the global headquarters of the combined BHP group, are located in Melbourne, Australia”

- Learn drug-gene product interactions from medical research literature

- Text classification for IE

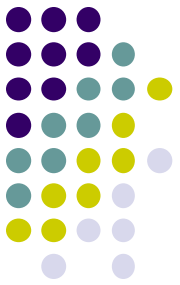
- Can be used to classify documents as “to be extracted” or not.
- In simple but compelling domains, this naïve technique is remarkably effective.

- **Subjectivity**

- Does the text convey factual or subjective content?

- **Stance classification**

- Given a set of possible positions, or stances, which is being taken by the author?



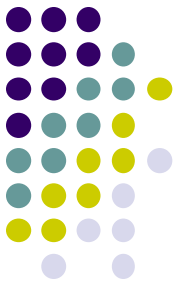
Word Sense Disambiguation

- **Word senses**

- Word sense is **one of the meanings of a word** in linguistics.
- Many words have multiple sense, or meanings, e.g., the verb **appeal** has the following senses:
 - Appeal take a course case to a higher court for review
 - Appeal, invoke request earnestly (something from someone)
 - Attract, appeal be attractive to

- **Word sense disambiguation**

- The problem of **identifying** the **intended word sense** in a given text
- **How can we tell living plants from manufacturing plants?**
 - “Town officials are hoping to attract new manufacturing plants through weakened environmental regulations.”
 - “The endangered plants play an important role in the local ecosystem.”
- **Ultimately, we need to understand the context.**



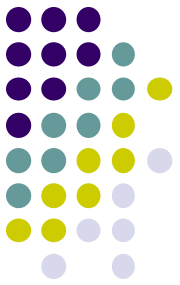
Harms in Sentiment Classifiers

- **Harms in sentiment classification**

- Kiritchenko and Mohammad (2018) found that most sentiment classifiers assign lower sentiment and more negative emotion to sentences with African American names in them.
 - This perpetuates negative stereotypes that associate African Americans with negative emotions.

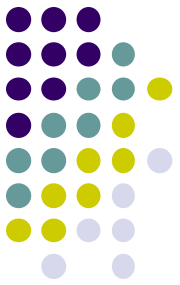
- **Toxicity detection**

- The task of detecting hate speech, abuse, harassment, or other kinds of toxic language
- But some toxicity classifiers incorrectly flag as being toxic sentences that are non-toxic but simply mention identities like blind people, women, or gay people.
- This could lead to censorship of discussion about these groups.



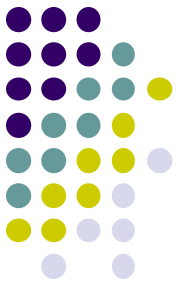
What Causes these Harms?

- **Can be caused by:**
 - Problems in the **training data**; **machine learning algorithms** are known to **amplify the biases** in their training data.
 - Problems in the human labels
 - Problems in the resources used (like lexicons)
 - Problems in model architecture (like what the model is trained to optimized)
- Mitigation of these harms is an open research area.
 - Meanwhile: **model cards** (Introduced by Google researchers)
 - Documentation tools for machine learning algorithms to provide essential information about a specific model, e.g., model's capabilities, intended use cases, limitations, evaluation metrics, developers, ethical considerations, future work, etc.
 - **Purpose:** transparency, accountability, informed decision-making



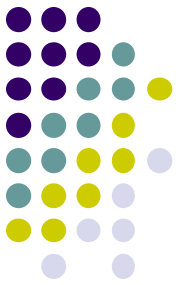
Text Classification Steps

- **Preprocess text data**
 - Tokenization, text normalization, etc.
- **Transform text data to numerical data (Vectorization)**
 - Word frequency
 - E.g., for sentiment analysis, count positive and negative words using a sentiment lexicon and classify in the text and classify based on majority (MLE).
 - **Limitations of word frequency in sentiment analysis**
 - Word ambiguity, sarcasm, and contextual meaning can distort results.
 - Modifiers (e.g., “I can’t stand, believe how great) affect sentiment interpretation
 - **More sophisticated methods**
 - Machine learning and Natural Language Processing techniques to create meaningful vector or matrix for machine learning algorithms
- **Apply classification algorithms**



Preprocessing Text Data for NLP Tasks

Preprocessing Text for NLP Tasks



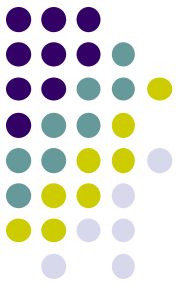
- **Preprocessing text data**

- Processing text data appropriate for machine learning algorithms
- Text normalization
 - The process of transforming text into a single **canonical form**.
- Vectorization
 - Transform text into vectors for model input

- **Text normalization process**

- 1) Tokenizing words (splitting text into tokens)
- 2) Standardization
 - Standardize case, punctuation, numbers, symbols, dates, encoding conversion, case conversion, removal or insertion of markup, etc.
 - Stemming and lemmatization to reduce words to root/base forms
- 3) Segmenting sentences
 - Sentence boundary detection

Regular Expressions



- **Regular expressions**

- Sophisticated sequences of regular expressions are often the first model for any text processing text.
- Tokenization in NLTK:

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     ([A-Z]\.)+          # abbreviations, e.g. U.S.A.
...     | \w+(-\w+)*         # words with optional internal hyphens
...     | \$?\d+(\.\d+)?%?    # currency and percentages, e.g. $12.40, 82%
...     | \.\.\.            # ellipsis
...     | [][.,;"'()?:-_']  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

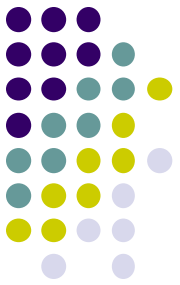
- **Scikit-learn packages for tokenization and vectorization**

- **CountVectorizer** from the package `sklearn.feature_extraction.text`

- **Machine learning classifiers**

- Can be used for hard text processing tasks
 - Regular expressions are still used for pre-processing, or as features in the classifiers.
- Can be very useful in capturing generalizations

Tokenization



- **Space-based tokenization**

- A very simple way for languages that use space characters between words: Arabic, Cyrillic, Greek, Latin, etc.

Given a text file, output the word tokens and their frequencies:

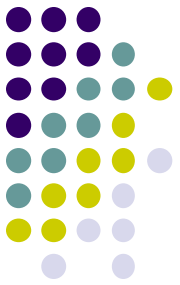
```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | uniq -c
```

- **Some issues in space-based tokenization**

- **Characters:** can't just blindly remove punctuation:
 - Ph.D., AT&T, \$45.55, 01/02/06, <http://www.fullerton.edu>, hashtags(#nlproc)
- **Clitic:** a word that doesn't stand on its own: **we're** (separate it to “we are”)
- **Multiword expressions:** **New York, rock 'n roll**
- **Languages without using spaces to separate the words** (Chinese, Thai)
 - Chinese words are composed of characters called “hanzi”. Each one represents a meaning unit. 姚明进入总决赛 “**Yao Ming reaches the finals**”
 - In Chinese it's common to just treat each character (zi) as a token. So, the segmentation step is simple.

- **Other options**

- White-space or single-character segmentation (space, tab, newlines etc.)
- Use the data to tell us how to tokenize (by exploring the text)
- **Subword tokenization** (because tokens can be parts of words as well as whole words)



Text Standardization

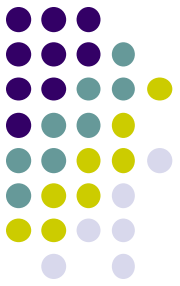
- **Case folding**

- Mapping everything to lower case.
 - Applications like information retrieval (IR) reduce all letters to lower case since users tend to use lower case and helpful for generalization.
- Possible exceptions
 - Upper case in mid-sentence, e.g., *General Motors*, *Fed* vs. *fed*, *SAIL* vs. *sail*
 - For sentiment analysis, machine translation, information extraction, case is helpful (e.g., *US* versus *us* is important).

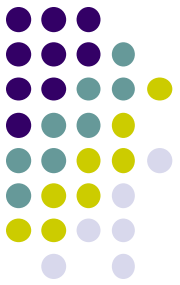
- **Word normalization**

- Putting words in a standard format
 - U.S.A. or USA, uhhuh or uh-huh, Fed or fed
 - am, is, be, are
- Removing stop words
 - **Stop words** are words that carry little to no significant meaning, e.g., “a”, “the”, “in”, “for”, “and”, “but”, “I”, “she”, etc.
- Stemming and lemmatization

Stemming

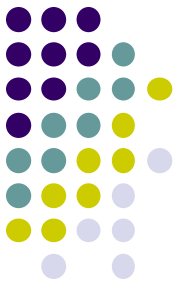


- Reducing terms to stems, chopping off affixes crudely
- Porter Stemmer (Stemming algorithm by Porter, 1980):
 - Based on a series of rewrite rules: a cascade, in which output of each pass fed to next pass
 - Example rules:
 - ATIONAL $\rightarrow$ ATE (e.g., relational $\rightarrow$ relate)
 - ING $\rightarrow$ ϵ if stem contains vowel (e.g., motoring $\rightarrow$ motor)
 - SSSES $\rightarrow$ SS (e.g., grasses $\rightarrow$ grass)
- Dealing with complex morphology is necessary for many languages.



Lemmatization

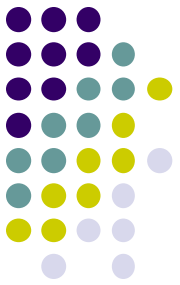
- **Determining if two words** have the **same root**, despite their surface differences
 - *am, are, is* → *be*
 - *car, cars, car's, cars'* → *car*
 - Spanish **quiero** ('I want'), **quieres** ('you want') → **querer** 'want'
 - *"He is reading detective stories"* → *"He be read detective story"*
- **Lemmatization is done by Morphological Parsing**
 - **Morphology** is the study of the way words are built up from smaller meaning-bearing units called **morphemes**.
 - Two broad classes of morphemes
 - **Stems**: The core meaning-bearing units
 - **Affixes**: Parts that adhere to stems, often with grammatical functions
 - **Morphological Parsers** parse "**cats**" into two morphemes "**cat**" and "**s**".



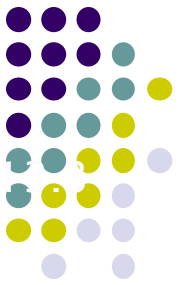
Sentence Segmentation

- Segmenting a text into sentences
- Useful cues for segmenting:
 - Characters “.”, “?”, “!”
 - “!” and “?” mostly unambiguous but period “.” is very ambiguous for a sentence boundary
 - Abbreviations like Inc. or Mr., numbers like .02% or 4.3
 - Sentence tokenization and word tokenization may address the problem jointly.
- Common algorithm
 - Sentence tokenization methods work by first deciding whether a period is part of the word or is a sentence-boundary marker using rules or machine learning.
 - An abbreviation dictionary can be helpful.

Additional Text Processing Tasks

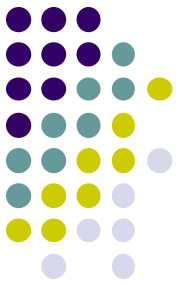


- Vocabulary size filtering
 - Limit vocabulary size to reduce model complexity as a small number of word types accounts for the majority of word tokens, e.g., keep words that occur $\geq x$ times, or in $\geq y\%$ of documents
- Handling unknown words in the training dataset
- Handling stop words
- Considering *word occurrence* regardless of its frequency
- Dealing with negation
- Handling small training data
 - How to deal with low frequency of sentiment bearing words



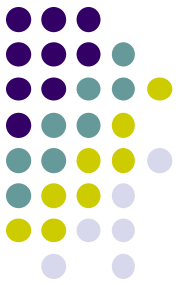
Unknown Words and Stop Words

- **What to do with unknown words in the training data but in a testing dataset?**
 - **Unknown words**, also called **Out-Of-Vocabulary (OOV) words**
 - **Simple idea:**
 - Ignore them by removing them from the test document, pretending they weren't there since it doesn't help.
 - Knowing which class has more unknown words is not generally helpful!
 - **More sophisticated methods**
 - E.g., replace them with a special token <UNK>, subword/characters
- **How about stop words** (e.g., “the”, “a”)?
 - Some systems ignore top certain number of stop words (from both training and testing sets) since they are very frequent.
 - But **removing stop words doesn't usually help.**
 - In practice most algorithms use all the words without using stop word lists.



Importance of Word Occurrence

- For sentiment analysis, **word occurrence** seems to be more important than **word frequency**.
 - The fact that a word occurs 5 times may not tell us much more.
 - But the occurrence of the word *fantastic* tells us a lot.
- How to incorporate word occurrence
 - Binary multinomial Naïve Bayes
 - A variant of the Multinomial Naïve Bayes
 - Transforming the feature set into binary values to indicate presence (1) or absence (0) in the document rather than its frequency => **Binary Multinomial Matrix**
 - **Note:** this is different than Bernoulli naive bayes.
 - **Algorithm:**
 - **Clip the word counts at 1** by removing all duplicate words **within** each document.
 - Create a DTM based on the clipped word count.
 - Then, compute the NB.



Example Binary Multinomial Matrix

- **Binarization is within document!**

- So, counts can still be more than one.

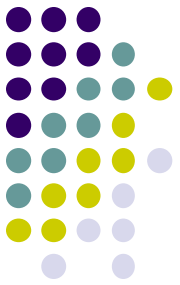
Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	–	+	–
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1



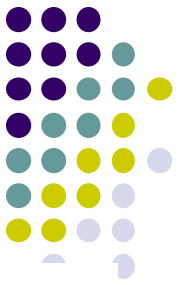
Dealing with Negation

- **Negation changes the sentiment of a word.**
 - “I really like this movie” vs “I really **don't** like this movie”
 - **Negation** changes the meaning of a **positive** word “like” to **negative**.
 - “**Don't dismiss** this film” vs “**Doesn't** let us get **bored**”
 - **Negation** can also change **negative** to **positive-ish**.
- **Simple baseline method: Negation Propagation**

(S. Das and M. Chen, 2001, B. Pang, L. Lee, and S. Vaithyanathan, 2002)

 - **Idea:** Define the negation scope using **sentiment bearing words**.
 - **Add** a **negation marker** (e.g., “**NOT_**” to the **affected words**)
 - **Avoid** adding the marker to non-sentiment bearing or neutral words.
 - Then, the negation marker is treated as a negative word.
 - Positive words with negation marker turn into negative.
 - Negative words with negation marker turn into positive.
 - Example: for “I **didn't** like this movie, but” =>
 - “I **didn't** **NOT_like** **NOT_this** **NOT_movie**, but” (up to a punctuation) **or**
 - “I **didn't** **NOT_like** this movie, **but**” (nearest sentiment bearing word)

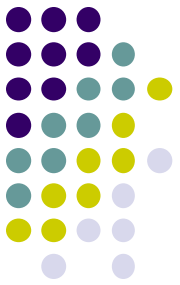
What If Training Data is Not Enough?



- **Issue: Low frequency of sentiment bearing words**
 - If training data is sparse or not representative of the test set.
- **Basic idea**
 - Use **pre-built word lists** called “**lexicons**”.
 - Various publicly available lexicons
 - E.g., for sentiment bearing words (SentiWordNet, AFINN, VADER, TextBlob, etc.)
- **MPQA Subjectivity Cues Lexicon**

(T. Wilson, J. Wiebe, and P. Hoffman 2005, Riloff and Wiebe 2003)

 - 6885 words from 8221 lemmas, annotated for intensity (strong/weak)
 - 2718 positive and 4912 negative
 - + : *admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great*
 - – : *awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate*
- **The general inquirer** (P. Stone, et al., 1966) with **various categories**
 - Positive (1915 words) and Negative (2291 words)
 - Strong vs Weak, Active vs Passive, Overstated vs Understated
 - Pleasure, Pain, Virtue, Vice, Motivation, Cognitive Orientation, etc.



How to Use Lexicons

1. Choose lexicon

- Sentiment lexicons, domain-specific lexicons, custom-built lexicons

2. Create features using lexicon

Match each word in the text to the words in the lexicon to add **new features** to the training data.

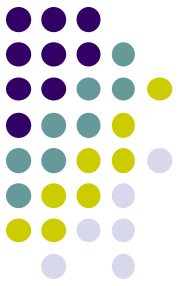
- **Count-based features**
 - Count or ratio of positive and negative words, presence/absence of specific word
- **Category tagging features**
 - Each word in the lexicon is associated with one or more categories
 - Count words belonging to each category using multi-label lexicons
 - Aggregate across words for document-level category features

3. Get sentiment scores for words

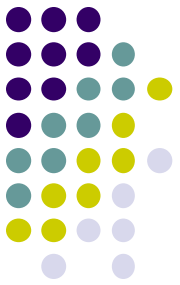
- Sentiment score = (positive word count) – (negative word count), ratio, or weighted score, probability, etc.

4. Aggregate scores

- Sum or average the sentiment scores for all words in the text to get an overall sentiment score for the document.

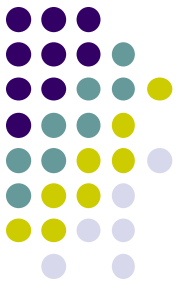


Vectorizing Text Data



Vectorizing Text Data

- **Vectorization of text data**
 - Converting text data to numerical data and stored in vectors
 - Why?
- **Methods for vectorizing text data**
 - One-hot encoding
 - Counting frequencies of words
 - Bag of Words (BoW), weighted frequencies (tf-idf), etc.
 - Embeddings
 - “**Embedding**” refers to a “**dense**” and “**low-dimensional representation**” of input data in a “**continuous vector space**” by capturing the original properties of data.
 - Learning vector representations of words that captures semantic or contextual relationships among words in text
 - One of the important tasks in NLP



One-Hot Encoding

the	quick	brown	fox	jumps	over	the	lazy	dog
1	0	0	0	0	0	1	0	0
0	1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	1

A vector that has one element = 1,
all others are zero.

or just unique vectors

Various forms of one-hot encoding

- **One-hot encoding**

- Each word is represented by a binary vector.

- **Strengths**

- Simple and useful for selecting column or row vectors of matrices

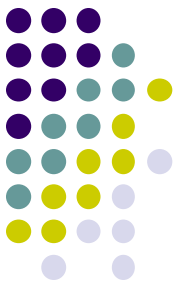
- **Problems**

- Loss of information as each word is treated independently without considering the relationships with others and simply represents its presence
- Increased dimension (the large # of words in the data)



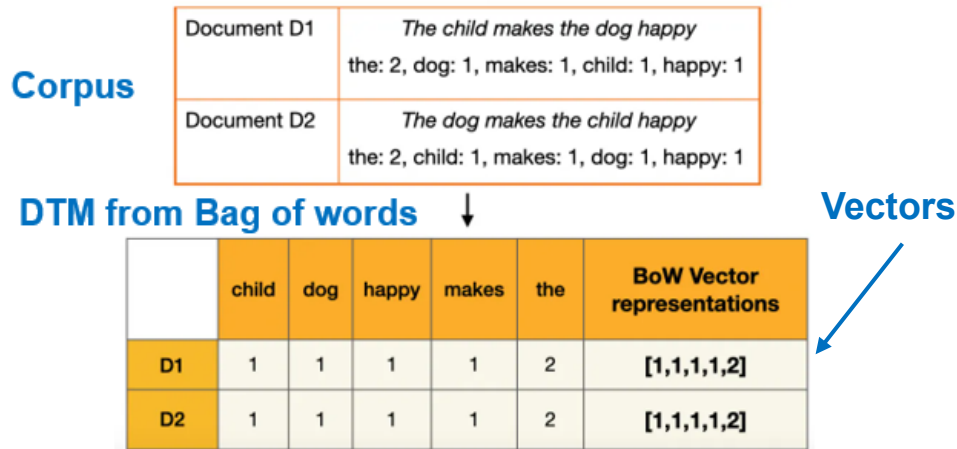
it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

- 39



Document-Term Matrix (DTM)

(or Term-Document Matrix)

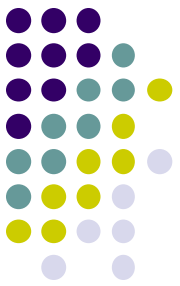


- **Bag of Words (BoW) in Document-Term Matrix (DTM)**

- Column indices represent document indices of corpus.
- Row indices represent term (word) indices of corpus.
- A column vector represents the frequencies for each word across the documents.
- A row vector represents the frequencies for each word in the document.

- **Usefulness of frequencies**

- Simple and useful, e.g., if *dog* appears a lot near *child*, that's useful information.⁴⁰



Vectors from Term-Document Matrix

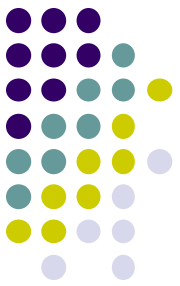
	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Transpose of
Document-Term Matrix,
 DTM^T

- A **document** as a vector of words
 - A compressed representation (encoding) of the document's content
 - When **two documents are similar**, the **corresponding vectors are similar**.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- A **word** as a vector of documents
 - A word vector encodes its document distribution
 - A **profile of documents, or possible word meaning**
 - The word **"battle"** occurs in **"Julius Caesar"** and **"Henry V"**
 - The word **"fool"** occurs in comedies, especially **"Twelfth Night"**



Term-Context Matrix

(Term-Term or Word-Word Matrix)

Context words (columns): $P(w, c) = \text{raw count}$

Words
(rows)

	computer	data	result	pie	sugar	<u>count(w)</u>
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
<u>count(context)</u>	4997	5673	473	512	61	<u>11716</u>

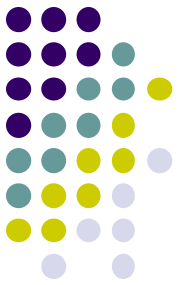
A word as a vector
of context words

Possible word
meaning

Word probabilities: $P(w, c) = \text{probability}$

	<u>p(w,context)</u>					<u>p(w)</u>
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
<u>p(context)</u>	0.4265	0.4842	0.0404	0.0437	0.0052	

- $P(\text{information, data}) = 3982/11716 = 0.3399$
- $P(\text{information}) = 7703/11716 = .6575$, $P(\text{data}) = 5673/11716 = .4842$



Example Sentiment Analysis

Movie review data

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun



How to vectorize
this data?

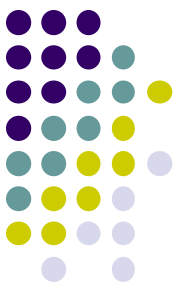
DTM?

- **Preprocessing**

- Vectorize the documents

- **Classification algorithms**

- Naïve Bayes, Logistic Regression, or Multilayer Perceptron (MLP)
 - Given a test example x we compute $P(y|x)$ using learned weights $\hat{w}$ and return whichever class y is higher probability by MLE.



Customized (curated) Vectors

An example review document

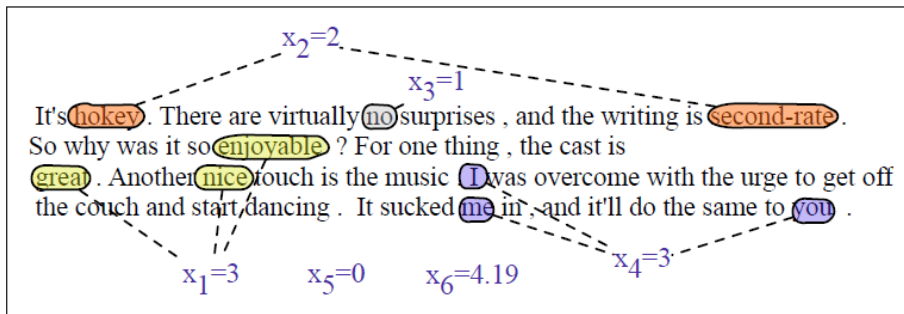


Figure 5.2 A sample mini test document showing the extracted features in the vector x .

A vectorized data for + example

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(66) = 4.19$

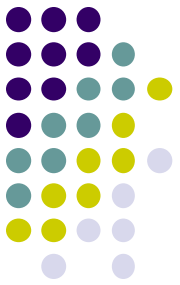
+

- **Preprocessing**

- Vectorize the documents based on manual/expert-driven feature design
 - Manual hand-crafted design to adapt features for a specific task/domain

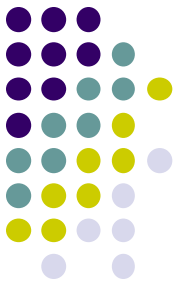
- **Classification**

- Using Naïve Bayes, Logistic Regression, or Multilayer Perceptron (MLP)



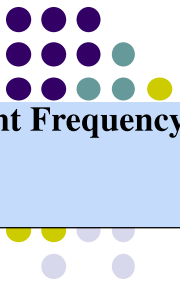
Difficulty of Counting Words

- “I do uh main- mainly business data processing”
 - Fragments, filled pauses
- “Seuss’s cat in the hat is different from other cats!”
 - Cat and cats are same lemma (same stem, part of speech)
 - Cat and cats are different wordforms (the full inflected surface form)
- “they lay back on the San Francisco grass and looked at the stars and their”.
 - Is 15 tokens or 14?
 - Token is an instance of that type in running text.
 - Is 13 types, 12, or 11?
 - Type is an element of the vocabulary (# of unique words that occur in the sentence, e.g., we only count “the” once).
- **Word count can be different from language to language.**



Raw Word Frequencies

- **Some problems of using raw frequencies**
 - Overly frequent words like *the*, *it*, or *they* are **not** very informative.
 - Two documents with different meaning can have the same BoW.
 - Some words are **evenly distributed** across many documents.
 - Some words may be **more important** than other words.
 - **Unable** to capture **the importance of a keyword**, **relationships**, or **semantics** within a document
- **Better ideas**
 - Word weighting
 - Capturing context => ultimately understanding the text content



	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Word Weighting

- **Different frequencies**

- Collection Frequency (total # of occurrences of the term in the entire corpus)
- Term Frequency (*tf*) (total # of occurrences of the term in a document)
- Document Frequency (*df*) (total # of documents that contain the term)

- **Assigning weights to words**

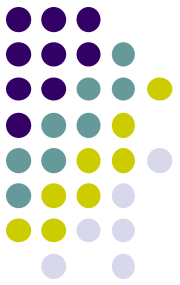
1. Remove **stop words** (insensitive to important words)
2. Count Term Frequency.
3. Normalize Term Frequency by document length for relative term frequency
 - Either dividing *tf* by total # of terms *n* in the document $\frac{tf}{n}$ or
 - Taking $\log_{10} tf$ is to control scaling effect of common words
4. Multiply *tf* by **log Inverse of Document Frequency** (*idf*): $tf \times idf$
 - To deal with the words evenly distributed across many documents
 - Both *tf* and *idf* are weights for a term.

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

Calculating Word Weights

- **Normalized Term Frequency:** $tf_{t,d} = \frac{count(t,d)}{n}$
 - Frequency of a term t in a document d with total n # of terms in d
 - or $tf_{t,d} = count(t,d)$ or $tf_{t,d} = \log[count(t,d) + 1]$
- **Inverse Document Frequency:** $idf_t = \log(\frac{N}{df_t})$
 - N is total # of documents in a corpus and df_t is # of documents containing term t .
 - More frequent its usage across documents, the lower its score.
- **TF-IDF** is a weighted term t in document d :

$$w_{t,d} = tf_{t,d} \times idf_t$$



Example: TF-IDF Weighted Data

- DTM based on raw counts

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- Each column represents a document (Shakespeare's play).
- Each row represents a word appeared in each document.

- Weighted frequency values:** $w_{t,d} = idf_t \times tf_{t,d}$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

- A high **tf-idf** value for a term within a document **suggests** that the term is both **important** within the document and **relatively unique** to that document.



Advantages and Limitations of TF-IDF

- **Advantages**

- It balances term frequency and global importance.
- Computationally efficient
- **Works well** for document ranking, information retrieval, text classification
 - Useful for efficient document search by keyword

- **Limitations**

- Ignore semantics and assume term independence
- So, not good enough for word prediction or generation that is important for many NLP tasks
- Sparse representation like DTM matrix
 - A large corpus → high-dimensionality (lots of words)
 - Vectors may be mostly zeros

- **Some improvement ideas**

- can we assign weights based on word association?
- Ideally, can we capture the context or meaning?



Word Association Weighting

- **Pointwise Mutual Information (PMI):**

$$PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)} \text{ (commonly log base 2)}$$

- $PMI(w, c)$ is the **joint probability** of **two words co-occurring**, given their individual frequencies **under the assumption of independent words**
- If $PMI(w, c) > 0$ (positive PMI)
 - Implies that the two terms co-occur **more frequently than by chance**.
 - This is **useful information**.
- If $PMI(w, c) < 0$ (negative PMI)
 - Negative PMI (when **$\log(<1)$**), is **not useful**, but we **already assumed the word independence**. **Isn't it less than expected?**
 - **How about PMI = 0?** (independence, so OK as we already assumed it)

- **Problems of PMI**

- PMI is biased toward rare words.
 - Higher scores to infrequent words that co-occur a few times, even if these co-occurrences are not necessarily significant.
- Any better way to deal with negative PMI?

Word Association Weighting



- **Positive PMI (PPMI):**

$$PPMI(w, c) = \max(PMI(w, c), 0)$$

- *PPMI* sets **negative PMI** to **zero** to captures only positive word associations and address the bias of rare words.

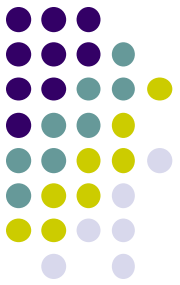
- **Shifted PPMI:** $PPMI^\alpha(w, c) = \max(PMI^\alpha(w, c), 0)$

- **What if** we want to give **rare words** slightly higher probabilities?

- **Shifted PMI** $PMI^\alpha(w, c) = \log \frac{P(w, c)}{P(w)P(c)^\alpha}$ where $P(c)^\alpha = \frac{\text{count}(c)^\alpha}{\sum_c \text{count}(c)}$
- This helps because for **rare** c , $P(c)^\alpha > p(c)$ where (e.g., $\alpha=0.75 < 1$).
- E.g., for two events, $p(a) = .99$, $p(b) = .01$, $P(a)^{.75} = \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = .97$, $P(b)^{.75} = .03$

- **Advantages and Limitations of PMI**

- Advantage
 - Considering word similarity. Useful for creating word-context matrices
- Limitation
 - Still sparse vector



Example PPMI on Term-Context Matrix

Probabilities in Term-Context Matrix

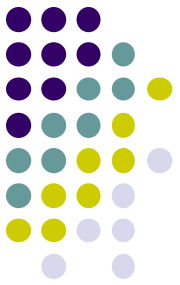
	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	<u>0.3399</u>	0.0323	0.0004	0.0011	<u>0.6575</u>
p(context)	0.4265	<u>0.4842</u>	0.0404	0.0437	0.0052	

PPMI matrix (negative values replaced by 0)

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	<u>0.09</u>	0.28	0	0

- $PPMI(\text{information}, \text{data}) = .3399 / (.6575 \times .4842) = 0.944$

Advantages and Limitations of Frequency-based Vectorization

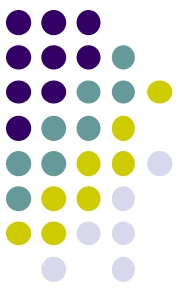


- **Advantages**

- Simple and easy to implement, no need for extensive preprocessing
- Flexibility in feature selection for dimensionality reduction
 - Like term frequency, inverse document frequency, etc.
- Effective for basic text classification
- Providing a good baseline

- **Limitations**

- **Sparse matrix representation** (lots of zeros)
 - Each document may have a **variable length representation** (due to the sequential representation of text) but the bag of words is a **fixed length representation**.
 - Many cells may contain zeros due to infrequent words
- **High dimensional matrix** (easily > 10,000 unique words)
 - results in large storage and computational complexity with large vocabularies
- **Not able to capture semantics**



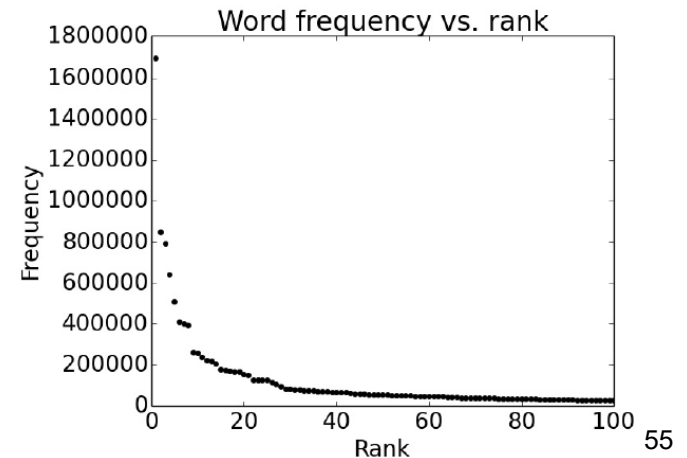
Sparsity of Text Data

any word	
Frequency	Type
1,698,599	the
849,256	of
793,731	to
640,257	and
508,560	in
407,638	that
400,467	is
394,778	a
263,040	I

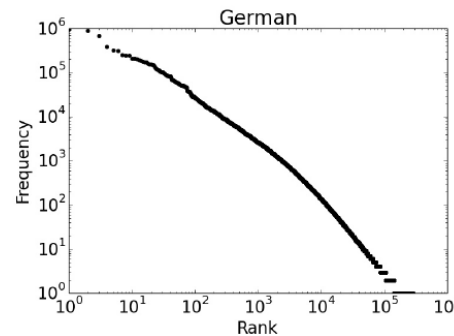
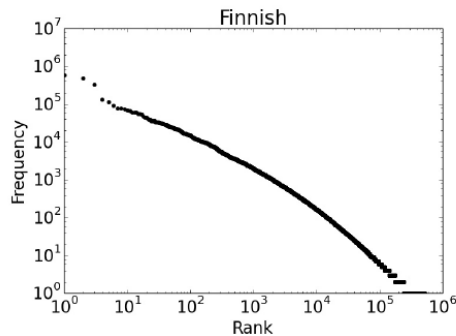
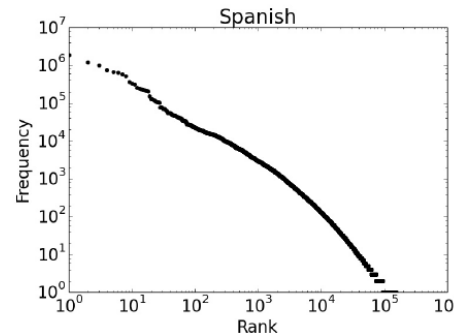
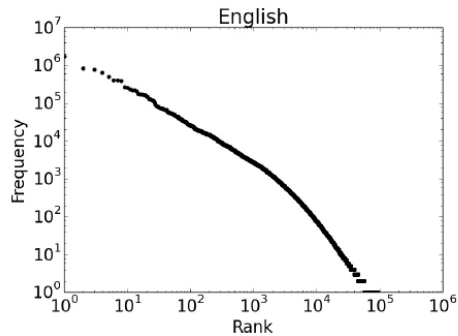
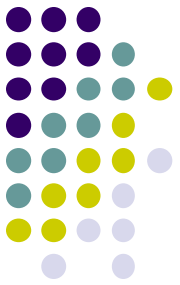
nouns	
Frequency	Type
124,598	European
104,325	Mr
92,195	Commission
66,781	President
62,867	Parliament
57,804	Union
53,683	report
53,547	Council
45,842	States

Frequency of words in the **English Europarl corpus** out of *24 millions word token*

- **Zipf's law: $f \times r \approx k$**
 - f = frequency of a word
 - r = rank of a word frequency
 - k = a constant
- **Word counts from a large text corpus**
 - **36231** out of **93638** distinct word types occur **only once**.

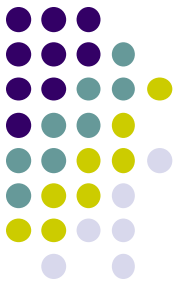


Word Frequency Ranks in Languages



● Implications of Zipf's Law

- Regardless of how large our corpus is, there will be **a lot of infrequent** (or zero-frequent) **words**.
 - In fact, the same holds for many other levels of linguistic structure (e.g., syntactic rules in a Context-Free Grammar).
- This means we **need to find clever ways to estimate probabilities** for things we have **rarely** or **never seen** during **training**.



How can we create a Vector that Capture Word Similarity/Context?

- **Word-Word or Term-Context Matrix**

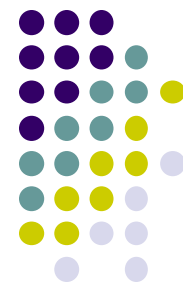
	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

- Two **words** are similar in meaning if their context vectors are similar.

is traditionally followed by	cherry	pie, a traditional dessert
often mixed, such as	strawberry	rhubarb pie. Apple pie
computer peripherals and personal	digital	assistants. These devices usually
a computer. This includes	information	available on the internet

- In addition, how can we create a **dense vector** (instead of sparse)?

Word Embedding



- **Word embedding**

- A **dense vector** representation of words that captures **contextual information** about words (semantic relationships) in text data
- **How can we create a dense vector for efficient representation?**
 - Learn co-occurrences of words or related words within a context window, **not every word**
 - Therefore, non-zero in all dimensions
 - Compress the information (by **encoding**)

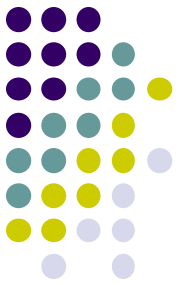
- **Embedding matrix E with dimensions (V, d) : $E = d \times |V|$**

- V : the size of vocabulary; d : the dimensionality of the embedding space (columns); the cell value E_{ij} represents the j -th component of the embedding vector (column) for the i -th word (row).
 - Each row represents the **embedding vector** (or each column if transposed).
 - Each column represents a **dimension** of the embedding space.

- **Capturing the contextual information and semantics**

- **Learning the probabilities** between words => **Essential component for NLP**.
 - **Language Modeling** (to be discussed in the next lecture)
- **Pretrained embeddings**: Embeddings previously learned from large corpora

Example Word Embedding

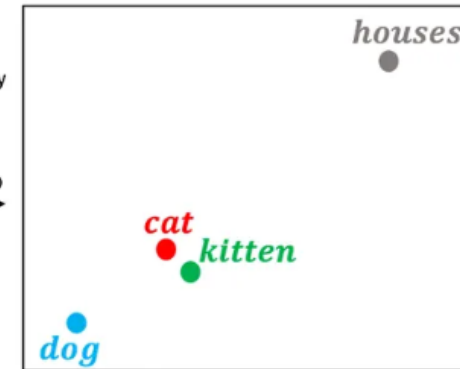


dimension

	living being	feline	human	gender	royalty	verb	plural
<i>cat</i> →	0.6	0.9	0.1	0.4	-0.7	-0.3	-0.2
<i>kitten</i> →	0.5	0.8	-0.1	0.2	-0.6	-0.5	-0.1
<i>dog</i> →	0.7	-0.1	0.4	0.3	-0.4	-0.1	-0.3
<i>houses</i> →	-0.8	-0.4	-0.5	0.1	-0.9	0.3	0.8

Dimensionality
reduction of
word
embeddings
from 7D to 2D

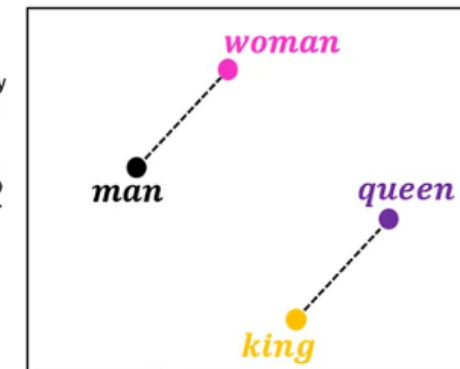
Words in continuous vector space

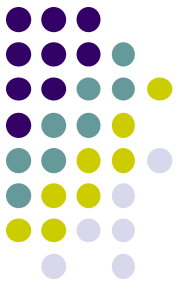


Vectors

<i>man</i> →	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7
<i>woman</i> →	0.7	0.3	0.9	-0.7	0.1	-0.5	-0.4
<i>king</i> →	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6
<i>queen</i> →	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9

Dimensionality
reduction of
word
embeddings
from 7D to 2D





Dense Word Embeddings

- **Low dimensionality**

- Dense vectors have a small dimensions (e.g., 50-1,000 instead of 100,000 in sparse vector)
- Each word is represented by a vector of (**predefined**) dimensions.
- Each dimension captures **some latent feature of the word** (not all the words).

- **Each element**

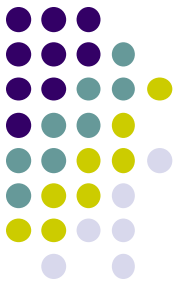
- **Any continuous values**, **unlikely zeros** as these are not necessarily frequency values but can be **values learned from training**

- **Capture semantic relationships**

- Embeddings are typically obtained through training process.
 - Some embedding methods: Word2Vec, LSA, Glove, FastText, ELMO, etc.

- **Efficient and compact**

- Smaller size of vectors



Pros and Cons of Embeddings

- **Pros**

- Captures semantics of words and contextual relationships
- Handles synonyms and analogies
- Reduce dimensionality – efficiency of memory and computation
- Adaptability to various languages – works across multiple languages
- Pretrained models available

- **Cons**

- Requires large amount of data
- Static embeddings lack context sensitivity
- Out-of-vocabulary issues – incomplete representation when dealing with domain specific terms or new words
- Bias in pre-trained embeddings
- High computational resources for training
- Fixed vocabulary size
 - In traditional embeddings, the vocabulary size is fixed once the model is trained. Adding new words requires retraining or fine-tuning the model (resource-intensive). May not work well in dynamic environments